

ydevARApp Session Log

Contents

Session Log - 02_SDK.md 作成	1
セッション概要	2
作成したドキュメント構成	2
主な作業内容	2
1. 初期ドキュメント作成	2
2. アプリ事例セクション追加	2
3. Graphviz対応	2
4. マルチユーザーAR / 永続化セクション追加	3
5. SceneKitセクション追加	3
6. セクション番号整理	3
技術的なポイント	3
カバーした主要機能	3
WWDC25新機能	3
次のステップ候補	4
備考	4
Session Log - TestFlight配布 & Wiki作成 & Swift機能解説	4
セッション概要	4
1. TestFlight配布	4
1.1 App Store Connect設定	4
1.2 Bundle ID登録 (Apple Developer)	4
1.3 証明書作成 (Xcode)	4
1.4 アーカイブ & アップロード	5
1.5 TestFlight設定	5
2. GitHubリポジトリ整備	5
2.1 コミット内容	5
2.2 追加されたファイル	5
2.3 README.md作成	6
3. GitHub Wiki作成	6
3.1 Wiki構成	6
3.2 アーキテクチャ図 (Graphviz)	6
4. Swift/iOS 技術解説	7
4.1 Core Data マイグレーション	7
4.2 Automatic Grammar Agreement (iOS 15+)	8
4.3 Markdown Parser (iOS 15+)	9
4.4 Markdown Extended Attributes (iOS 15+)	9
5. 成果物まとめ	10
6. 備考	10

Session Log - 02_SDK.md 作成

日付: 2026-01-19 プロジェクト: ydevARApp 成果物: /docs/02_SDK.md

セッション概要

ydev勉強会向けに、ARKit + RealityKit + SceneKit の最新SDK機能を網羅したリファレンスドキュメントを作成。

作成したドキュメント構成

02_SDK.md (2000)

- 1.
 - 2.
 3. SceneKit SCN
 4. RealityKit
 - 5.
 - 6.
 7. (RealityKit 4 / WWDC25)
 - 8.
 - 9.
 10.
 - 10.1
 - 10.2
 - 10.3
 - 10.4
 - 10.5 Scene Geometry / LiDAR
 - 10.6 People Occlusion
 - 10.7
 - 10.8 Spatial Audio / 3D
 - 10.9 visionOS
 - 10.10 AR /
 - 10.11
 - 10.12
 - 11.
-

主な作業内容

1. 初期ドキュメント作成

- ARKit / RealityKit の機能一覧を表形式で整理
- 実装ベースの具体的な選択肢を提示
- コード例を各セクションに追加

2. アプリ事例セクション追加

- iOS ARアプリ / visionOSアプリの具体例を追加
- フレームワーク機能別にアプリを分類
- 「何が作れるか」という観点で整理

3. Graphviz対応

- ASCII図をGraphviz (DOT言語) 形式に変換
- 以下の図を作成:
 - ARKitトラッキング階層図
 - Scene Geometry処理フロー
 - RealityKit ECSアーキテクチャ
 - ARSession/RealityView選択フローチャート
 - SceneKit vs RealityKit選択フロー

- マルチユーザーAR構成図
- SharePlay統合図

4. マルチユーザーAR / 永続化セクション追加

- ARWorldMap (保存/復元)
- Collaborative Session
- MultipeerConnectivity
- SynchronizationComponent
- SharePlay統合
- 実装コード例

5. SceneKitセクション追加

- SceneKit vs RealityKit 比較表
- ARSCNView実装パターン
- SCNNode / SCNGeometry / SCNAction
- 物理演算 (SCNPhysicsBody)
- パーティクル (SCNParticleSystem)
- マテリアル設定
- モデル読み込み
- ヒットテスト

6. セクション番号整理

- SceneKit追加に伴う全セクション番号の再付番
- サブセクション (10.1~10.12) の番号修正

技術的なポイント

カバーした主要機能

カテゴリ	機能
トラッキング	平面/画像/顔/ボディ/ハンド/ルーム/オブジェクト
深度・メッシュ	Scene Geometry, Depth API, LiDAR
レンダリング	SceneKit, RealityKit ECS, マテリアル
インタラクション	ジェスチャー, 物理演算, 当たり判定
オーディオ	Spatial Audio, Ambient Audio
マルチユーザー	Collaborative Session, MultipeerConnectivity
永続化	ARWorldMap, CloudKit連携
visionOS	Hand Tracking, Room Tracking, Portal

WWDC25新機能

- SpatialTrackingSession
- EnvironmentBlendingComponent
- MeshInstancesComponent
- ManipulationComponent
- Low-Level Mesh / Texture API

次のステップ候補

- 01_Overview.md（プロジェクト概要）の作成
 - 03_Architecture.md（アーキテクチャ設計）の作成
 - サンプルコードプロジェクトの作成
 - Keynoteスライド用の図版エクスポート
-

備考

- ドキュメントはGraphviz図を含むため、レンダリングにはGraphviz対応ビューアが必要
 - コード例はSwift 5.9+ / iOS 17+ / visionOS 1.0+ を想定
-

Session Log - TestFlight配布 & Wiki作成 & Swift機能解説

日付: 2026-01-17 ~ 2026-01-19 プロジェクト: ydevARApp 成果物: TestFlight配布、GitHub Wiki、技術ドキュメント

セッション概要

ydevARAppのTestFlight配布準備、GitHubリポジトリ整備、Wiki作成、およびSwift/iOSの技術機能解説を実施。

1. TestFlight配布

1.1 App Store Connect設定

1. <https://appstoreconnect.apple.com>
2. App → + → App
3. :
 - : iOS
 - : ydevARApp
 - :
 - ID: com.dsgarage.ydevARApp
 - SKU: ydevarapp2026

1.2 Bundle ID登録 (Apple Developer)

1. <https://developer.apple.com/account/resources/identifiers/list>
2. + → App IDs → App
3. :
 - Description: ydevARApp
 - Bundle ID: Explicit → com.dsgarage.ydevARApp
4. Capabilities: Access WiFi Information
5. Continue → Register

1.3 証明書作成 (Xcode)

// Xcode

1. Xcode → Settings (Cmd + ,)
2. Accounts
3. Apple ID → Manage Certificates...
4. + → Apple Distribution

1.4 アーカイブ & アップロード

1. : Any iOS Device (arm64)
2. Product → Archive
3. Organizer → Distribute App
4. App Store Connect → Upload
5. → Upload

1.5 TestFlight設定

1. App Store Connect → → TestFlight
2. 5 30
3. : →
4. → →

TestFlightリンク: <https://testflight.apple.com/join/P8zMe1u3>

2. GitHubリポジトリ整備

2.1 コミット内容

```
git add ydevARApp.xcodeproj/project.pbxproj \  
ydevARApp/Assets.xcassets/AppIcon.appiconset/ \  
ydevARApp/ContentView.swift \  
ydevARApp/AR/ \  
ydevARApp/Models/ \  
ydevARApp/Networking/ \  
ydevARApp/Views/ \  
Info.plist \  
docs/
```

```
git commit -m "Add AR collaboration features with plane detection, spatial sharing, and object placement"  
git push origin main
```

2.2 追加されたファイル

ディレクトリ	ファイル	説明
ydevARApp/AR/	PlaneDetectionManager.swift	平面検知
	SpatialRecognitionManager.swift	空間認識
	SpatialSharingManager.swift	空間共有
	AvatarManager.swift	参加者アバター
	ObjectPlacementManager.swift	オブジェクト配置
	OcclusionManager.swift	オクルージョン
ydevARApp/Networking/	ObjectDetectionManager.swift	オブジェクト検知
	MultipeerManager.swift	P2P通信
ydevARApp/Models/	ParticipantInfo.swift	参加者情報
	SharedObject.swift	共有オブジェクト
ydevARApp/Views/	ARViewContainer.swift	ARView統合
	ConnectionStatusView.swift	接続状態UI
	ObjectPaletteView.swift	オブジェクト選択
	ShutterButtonView.swift	シャッターボタン

2.3 README.md作成

```
# ydevARApp

[yidev      ](https://yidev4.connpass.com/event/377507/)  AR

## TestFlight
[TestFlight  ](https://testflight.apple.com/join/P8zMe1u3)

##
-
- 3D
-   MultipeerConnectivity Collaborative Session
-   LiDAR
-
```

3. GitHub Wiki作成

3.1 Wiki構成

```
ydevARApp.wiki/
  Home.md          #      +
  PlaneDetectionManager.md  #
  SpatialRecognitionManager.md #
  SpatialSharingManager.md  #
  AvatarManager.md    #
  ObjectPlacementManager.md #
  OcclusionManager.md  #
  ObjectDetectionManager.md #
  MultipeerManager.md  # P2P
  ARViewContainer.md   # ARView
  Models.md           #
  images/
    architecture.dot  # Graphviz
    architecture.png   #
```

3.2 アーキテクチャ図 (Graphviz)

```
digraph ydevARApp {
    rankdir=TB;
    node [shape=box, style="rounded, filled", fontname="Helvetica"];

    // UI Layer
    subgraph cluster_ui {
        label="UI Layer";
        bgcolor="#BBDEFB";
        ContentView; ConnectionStatusView; ObjectPaletteView; ShutterButtonView;
    }

    // AR Container
    subgraph cluster_arcontainer {
        label="AR Container";
        bgcolor="#C8E6C9";
        ARViewContainer; ARView; Coordinator;
    }
}
```

```

// AR Managers
subgraph cluster_ar {
    label="AR Managers";
    bgcolor="#FFF9C4";
    PlaneDetectionManager; SpatialRecognitionManager;
    SpatialSharingManager; AvatarManager;
    ObjectPlacementManager; OcclusionManager; ObjectDetectionManager;
}

// Networking
subgraph cluster_network {
    label="Networking";
    bgcolor="#FFCCBC";
    MultipeerManager;
}

// Connections
ContentView -> ARViewContainer;
Coordinator -> PlaneDetectionManager;
SpatialSharingManager -> MultipeerManager [label=" "];
AvatarManager -> MultipeerManager [label=" "];
}

```

Wiki URL: <https://github.com/dsgarage/ydevARApp/wiki>

4. Swift/iOS 技術解説

4.1 Core Data マイグレーション

軽量マイグレーション設定

```

let container = NSPersistentContainer(name: "MyApp")
let description = container.persistentStoreDescriptions.first
description?.setOption(true as NSNumber,
    forKey: NSMigratePersistentStoresAutomaticallyOption)
description?.setOption(true as NSNumber,
    forKey: NSInferMappingModelAutomaticallyOption)

```

SwiftData マイグレーション (iOS 17+)

```

import SwiftData

enum SchemaV1: VersionedSchema {
    static var versionIdentifier = Schema.Version(1, 0, 0)
    static var models: [any PersistentModel.Type] { [Item.self] }

    @Model class Item {
        var name: String
        var timestamp: Date
    }
}

enum SchemaV2: VersionedSchema {
    static var versionIdentifier = Schema.Version(2, 0, 0)
    static var models: [any PersistentModel.Type] { [Item.self] }
}

```

```

@Model class Item {
    var name: String
    var title: String //
    var timestamp: Date
}

enum MyMigrationPlan: SchemaMigrationPlan {
    static var schemas: [any VersionedSchema.Type] {
        [SchemaV1.self, SchemaV2.self]
    }

    static var stages: [MigrationStage] {
        [MigrationStage.lightweight(fromVersion: SchemaV1.self, toVersion: SchemaV2.self)]
    }
}

```

4.2 Automatic Grammar Agreement (iOS 15+)

基本構文

```

//
let count = 3
Text("^\\(count) item](inflect: true)")
// 1 → "1 item", 3 → "3 items"

//
var morphology = Morphology()
morphology.grammaticalGender = .feminine

```

agreeWithArgument (iOS 17+)

```

//
"^([This])(agreeWithArgument: 1) ^\\(count) item](inflect: true) ^[is](agreeWithArgument: 1) ready"
// count=1 → "This 1 item is ready"
// count=3 → "These 3 items are ready"

//
"^([El])(agreeWithArgument: 1) ^[usuario](agreeWithArgument: 1) está ^[contento](agreeWithArgument: 1)"
// → "El usuario está contento"
// → "La usuaria está contenta"

```

従来のスペイン語対応 (agreeWithArgument以前)

```

// 1:
"welcome_male" = "Bienvenido"
"welcome_female" = "Bienvenida"

// 2: stringsdict
// NSStringPluralRuleType

// 3:
"user_is_happy" = "%@ usuari%@ está content%@"
String(format: format, "El", "o", "o") //
String(format: format, "La", "a", "a") //

```



```
// 4:
"welcome_user" = "Te damos la bienvenida"
```

4.3 Markdown Parser (iOS 15+)

AttributedString でパース

```
import Foundation

let markdown = "** ** * * ` `\"
let attributed = try! AttributedString(markdown: markdown)

// SwiftUI
Text(attributed)
```

パースオプション

```
let options = AttributedString.MarkdownParsingOptions(
    allowsExtendedAttributes: true,
    interpretedSyntax: .inlineOnlyPreservingWhitespace
)
```

SwiftUIでの使用

```
Text("**Welcome** to *SwiftUI*!")
Text("[Apple](https://apple.com)  ")
```

4.4 Markdown Extended Attributes (iOS 15+)

構文

```
^[ ](key: 'value')
^[ ](key1: 'v1', key2: 'v2')
```

カスタム属性の定義

```
//
enum PriorityAttribute: CodableAttributedStringKey,
    MarkdownDecodableAttributedStringKey {
    typealias Value = String
    static let name = "priority"
}

// AttributeScope
extension AttributeScopes {
    struct MyAppAttributes: AttributeScope {
        let priority: PriorityAttribute
        let swiftUI: SwiftUIAttributes
    }
    var myApp: MyAppAttributes.Type { MyAppAttributes.self }
}

//
let markdown = "^[ ](priority: 'high')\"
let options = AttributedString.MarkdownParsingOptions(
    allowsExtendedAttributes: true
)
let attributed = try! AttributedString(
```

```

        markdown: markdown,
        including: AttributeScopes.MyAppAttributes.self,
        options: options
    )

    //
    for run in attributed.runs {
        if let priority = run.priority {
            print("Priority: \(priority)")
        }
    }
}

```

5. 成果物まとめ

成果物	URL/パス
TestFlight	https://testflight.apple.com/join/P8zMe1u3
GitHub	https://github.com/dsgarage/ydevARApp
Wiki	https://github.com/dsgarage/ydevARApp/wiki
勉強会	https://yidev4.connpass.com/event/377507/

6. 備考

- macOS 26ではキーチェーンアクセスが「パスワード」アプリに統合
 - `open -a "Keychain Access"` で従来のアプリを起動可能
- Vision Pro警告 (UIRequiredDeviceCapabilities: arkit) は無視してOK
- 輸出コンプライアンス: HTTPSのみなら「暗号化なし」でOK